

ASSIGN1

August 8, 2026

```
[2]: #ASSIGNMENT 1
      #Python Data Structures (Lists)

      # Roll Number
      roll_number = "1024170414"

      # Create list L by multiplying each digit by 10
      L= [int(digit)*10 for digit in roll_number]
      print("list:",L)
```

list: [10, 0, 20, 40, 10, 70, 0, 40, 10, 40]

```
[3]: # . Add two numbers Using append()
      L.append(60)
      print("After append:",L)
```

After append: [10, 0, 20, 40, 10, 70, 0, 40, 10, 40, 60]

```
[4]: # Using insert()
      L.insert(3, 25)
      print("After insert(3, 25):", L)
```

After insert(3, 25): [10, 0, 20, 25, 40, 10, 70, 0, 40, 10, 40, 60]

```
[13]: # Remove two elements Using remove()
      L.remove(10)
      print("After remove(10):", L)
```

After remove(10): [70, 40, 40, 40, 25, 10, 10, 0, 0]

```
[7]: # Using pop()
      removed = L.pop(2)
      print("After pop(2):", L)
      print("Element removed using pop():", removed)
```

After pop(2): [10, 0, 25, 40, 10, 70, 0, 40, 10, 40]
Element removed using pop(): 20

```
[8]: #Sort the list Ascending order  
L.sort()  
print("Ascending Order:", L)
```

Ascending Order: [0, 0, 10, 10, 10, 25, 40, 40, 40, 70]

```
[9]: # Descending order  
L.sort(reverse=True)  
print("Descending Order:", L)
```

Descending Order: [70, 40, 40, 40, 25, 10, 10, 10, 0, 0]

```
[10]: # Slicing  
print("First three elements:", L[:3])  
print("Last three elements:", L[-3:])
```

First three elements: [70, 40, 40]

Last three elements: [10, 0, 0]

```
[11]: # List comprehension  
average = sum(L) / len(L)  
greater_than_average = [x for x in L if x > average]  
  
print("\nAverage of L:", average)  
print("Elements greater than average:", greater_than_average)
```

Average of L: 24.5

Elements greater than average: [70, 40, 40, 40, 25]

```
[2]: # Tuples  
  
# Tuple created using the first 8 values from List L  
scores = (10, 0, 20, 40, 10, 70, 0, 40)  
print("Scores Tuple:", scores)
```

Scores Tuple: (10, 0, 20, 40, 10, 70, 0, 40)

```
[7]: # Highest score and its index  
highest = max(scores)  
highest_index = scores.index(highest)  
print("\nHighest Score:", highest)  
print("Index of Highest Score:", highest_index)
```

Highest Score: 70

Index of Highest Score: 5

```
[8]: # Lowest score and its count
lowest = min(scores)
lowest_count = scores.count(lowest)
print("Lowest Score:", lowest)
print("Number of times Lowest Score appears:", lowest_count)
```

Lowest Score: 0

Number of times Lowest Score appears: 2

```
[9]: #Reverse the tuple and return as a list
reversed_list = list(scores[::-1])
print("\nReversed Tuple as List:", reversed_list)
```

Reversed Tuple as List: [40, 0, 70, 10, 40, 20, 0, 10]

```
[10]: #Search for a user-entered score
search = int(input("\nEnter a score to search: "))

if search in scores:
    print("First occurrence index:", scores.index(search))
else:
    print("Score not present in the tuple.")
```

Enter a score to search: 20

First occurrence index: 2

```
[11]: #Attempt to modify the tuple
try:
    scores[0] = 100
except TypeError as e:
    print("\nError:", e)
```

Error: 'tuple' object does not support item assignment

```
[12]: # Tuple unpacking using *
first_score, second_score, *remaining_scores = scores

print("\nFirst Score:", first_score)
print("Second Score:", second_score)
print("Remaining Scores:", remaining_scores)
```

First Score: 10

Second Score: 0

Remaining Scores: [20, 40, 10, 70, 0, 40]

```
[14]: # Random Numbers and Lists
import random

# Set random seed using your roll number
random.seed(1024170414)
# Generate a list of 100 random numbers between 100 and 900
numbers = [random.randint(100, 900) for _ in range(100)]

print("Random List:")
print(numbers)
```

Random List:

[731, 276, 399, 283, 223, 716, 323, 138, 619, 396, 613, 407, 708, 314, 804, 659, 208, 425, 194, 447, 561, 764, 575, 833, 580, 242, 234, 328, 368, 149, 271, 716, 844, 887, 419, 334, 396, 760, 425, 323, 657, 729, 242, 878, 155, 799, 460, 668, 106, 178, 404, 343, 571, 178, 256, 200, 669, 488, 394, 696, 796, 181, 129, 100, 562, 524, 312, 866, 853, 525, 424, 590, 272, 431, 867, 553, 783, 675, 426, 626, 196, 469, 804, 118, 126, 226, 630, 635, 338, 552, 520, 572, 572, 897, 753, 591, 190, 864, 630, 288]

```
[15]: #Count and print all odd numbers

odd_numbers = [num for num in numbers if num % 2 != 0]
print("\nNumber of Odd Numbers:", len(odd_numbers))
print("Odd Numbers:")
print(odd_numbers)

#Count and print all even numbers
even_numbers = [num for num in numbers if num % 2 == 0]
print("\nNumber of Even Numbers:", len(even_numbers))
print("Even Numbers:")
print(even_numbers)

#Count and print all prime numbers Function to check prime
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

Number of Odd Numbers: 41

Odd Numbers:

[731, 399, 283, 223, 323, 619, 613, 407, 659, 425, 447, 561, 575, 833, 149, 271, 887, 419, 425, 323, 657, 729, 155, 799, 343, 571, 669, 181, 129, 853, 525, 431, 867, 553, 783, 675, 469, 635, 897, 753, 591]

Number of Even Numbers: 59

Even Numbers:

[276, 716, 138, 396, 708, 314, 804, 208, 194, 764, 580, 242, 234, 328, 368, 716, 844, 334, 396, 760, 242, 878, 460, 668, 106, 178, 404, 178, 256, 200, 488, 394, 696, 796, 100, 562, 524, 312, 866, 424, 590, 272, 426, 626, 196, 804, 118, 126, 226, 630, 338, 552, 520, 572, 572, 190, 864, 630, 288]

```
[16]: # List comprehension for prime numbers
prime_numbers = [num for num in numbers if is_prime(num)]

print("\nNumber of Prime Numbers:", len(prime_numbers))
print("Prime Numbers:")
print(prime_numbers)
```

Number of Prime Numbers: 13

Prime Numbers:

[283, 223, 619, 613, 659, 149, 271, 887, 419, 571, 181, 853, 431]

```
[17]: # Find the most frequent number
most_frequent = max(set(numbers), key=numbers.count)
frequency = numbers.count(most_frequent)

print("\nMost Frequent Number:", most_frequent)
print("Frequency:", frequency)
```

Most Frequent Number: 572

Frequency: 2

```
[1]: #Sets
# Roll number digits
digits = [1, 0, 2, 4, 1, 7, 0, 4, 1, 4]

# Create sets A and B
A = {digit * 7 for digit in digits}
B = {digit * 9 for digit in digits}

print("Set A:", A)
print("Set B:", B)
```

Set A: {0, 7, 14, 49, 28}

Set B: {0, 36, 9, 18, 63}

```
[2]: # Union of A and B
union = A.union(B)
print("\nUnion of A and B:", union)
```

Union of A and B: {0, 36, 7, 9, 14, 49, 18, 28, 63}

```
[3]: # Intersection of A and B
intersection = A.intersection(B)
print("Intersection of A and B:", intersection)
```

Intersection of A and B: {0}

```
[4]: # Difference
A_difference_B = A.difference(B)
B_difference_A = B.difference(A)

print("A - B:", A_difference_B)
print("B - A:", B_difference_A)
```

A - B: {49, 28, 14, 7}

B - A: {9, 18, 36, 63}

```
[5]: # difference() gives values present in one set but not the other.
# symmetric_difference() gives all values that are not common to both sets.
```

```
[6]: # Symmetric difference
symmetric = A.symmetric_difference(B)
print("Symmetric Difference:", symmetric)
```

Symmetric Difference: {7, 9, 14, 18, 28, 36, 49, 63}

```
[7]: # Check subset and superset
print("\nIs A a subset of B?", A.issubset(B))
print("Is B a superset of A?", B.issuperset(A))
```

Is A a subset of B? False

Is B a superset of A? False

```
[8]: #Ask user for a value
X = int(input("\nEnter a value to remove from Set A: "))

A.discard(X)
print("Set A after discard:", A)
```

Enter a value to remove from Set A: 49

Set A after discard: {0, 7, 14, 28}

```
[9]: #Dictionaries
# Original dictionary
my_dict = {
    "name": "Ritika",
    "roll_no": "1024170414",
    "branch": "CSE",
    "age": 20,
    "city": "Patiala"
}

print("Original Dictionary:")
print(my_dict)
```

Original Dictionary:
{'name': 'Ritika', 'roll_no': '1024170414', 'branch': 'CSE', 'age': 20, 'city': 'Patiala'}

```
[10]: # Rename "city" to "location"
my_dict["location"] = my_dict.pop("city")

print("\nAfter renaming city to location:")
print(my_dict)
```

After renaming city to location:
{'name': 'Ritika', 'roll_no': '1024170414', 'branch': 'CSE', 'age': 20, 'location': 'Patiala'}

```
[11]: # Add CGPA
my_dict["cgpa"] = 8.5

print("\nAfter adding CGPA:")
print(my_dict)
```

After adding CGPA:
{'name': 'Ritika', 'roll_no': '1024170414', 'branch': 'CSE', 'age': 20, 'location': 'Patiala', 'cgpa': 8.5}

```
[12]: # Increase age by 1
my_dict["age"] = my_dict["age"] + 1

print("\nAfter increasing age by 1:")
print(my_dict)
```

After increasing age by 1:
{'name': 'Ritika', 'roll_no': '1024170414', 'branch': 'CSE', 'age': 21, 'location': 'Patiala', 'cgpa': 8.5}

```
'location': 'Patiala', 'cgpa': 8.5}
```

```
[13]: # Delete "branch" using pop()
dict_pop = my_dict.copy()

removed_branch = dict_pop.pop("branch")

print("\nAfter deleting branch using pop():")
print(dict_pop)
print("Value returned by pop():", removed_branch)
```

After deleting branch using pop():

```
{'name': 'Ritika', 'roll_no': '1024170414', 'age': 21, 'location': 'Patiala',
'cgpa': 8.5}
```

Value returned by pop(): CSE

```
[14]: # Delete "branch" using del
dict_del = my_dict.copy()

del dict_del["branch"]

print("\nAfter deleting branch using del:")
print(dict_del)
```

After deleting branch using del:

```
{'name': 'Ritika', 'roll_no': '1024170414', 'age': 21, 'location': 'Patiala',
'cgpa': 8.5}
```

```
[15]: # pop() removes and returns the deleted value,
# while del removes the key-value pair but does not return it.
```

```
[16]: # Iterate using items()
print("\nKey → Value:")

for key, value in my_dict.items():
    print(key, "→", value)
```

Key → Value:

name → Ritika

roll_no → 1024170414

branch → CSE

age → 21

location → Patiala

cgpa → 8.5

```
[17]: # Check whether "email" exists
print("\nChecking for email:")

if "email" in my_dict:
    print("Email:", my_dict["email"])
else:
    print("Email is not present in the dictionary.")
```

Checking for email:
Email is not present in the dictionary.

```
[18]: # Create friend's dictionary
friend_dict = {
    "name": "Priyancy",
    "roll_no": "1024031064",
    "branch": "CSE",
    "age": 19,
    "city": "Chandigarh"
}
```

```
[19]: merged_dict = {**my_dict, **friend_dict}

print("\nFriend Dictionary:")
print(friend_dict)

print("\nMerged Dictionary:")
print(merged_dict)
```

Friend Dictionary:
{'name': 'Priyancy', 'roll_no': '1024031064', 'branch': 'CSE', 'age': 19, 'city': 'Chandigarh'}

Merged Dictionary:
{'name': 'Priyancy', 'roll_no': '1024031064', 'branch': 'CSE', 'age': 19, 'location': 'Patiala', 'cgpa': 8.5, 'city': 'Chandigarh'}

```
[20]: #Dictionary comprehension
string_values = {
    key: value
    for key, value in my_dict.items()
    if isinstance(value, str)
}

print("\nDictionary containing only string values:")
print(string_values)
```

Dictionary containing only string values:

```
{'name': 'Ritika', 'roll_no': '1024170414', 'branch': 'CSE', 'location':  
'Patiala'}
```

[]: